

Two Heads are Better than One: Robust Learning Meets Multi-branch Models

Zongyuan Zhang[†], Qingwen Bu[‡], Tianyang Duan[†], Zheng Lin[§], Yuhao Qing[†],
Zihan Fang[¶], Heming Cui[†], Dong Huang[#]

[†] Department of Computer Science, The University of Hong Kong, Hong Kong, China.

[‡] Department of Electronic Engineering, Shanghai Jiao Tong University, Shanghai, China.

[§] School of Computer Science, Fudan University, Shanghai, China.

[¶] Department of Computer Science, City University of Hong Kong, Hong Kong, China.

[#] School of Computing, National University of Singapore, Singapore.

Abstract—Deep neural networks (DNNs) are vulnerable to adversarial examples, in which DNNs are misled to false outputs due to inputs containing imperceptible perturbations. Adversarial training, a reliable and effective method of defense, may significantly reduce the vulnerability of neural networks and becomes the de facto standard for robust learning. While many recent works practice the data-centric philosophy, such as how to generate better adversarial examples or use generative models to produce additional training data, we look back to the models themselves and revisit the adversarial robustness from the perspective of deep feature distribution as an insightful complementarity. In this paper, we propose *Branch Orthogonality adveRsarial Training* (BORT) to obtain state-of-the-art performance with solely the original dataset for adversarial training. To practice our design idea of integrating multiple orthogonal solution spaces, we leverage a simple and straightforward multi-branch neural network that eclipses adversarial attacks with no increase in inference time. We heuristically propose a corresponding loss function, branch-orthogonal loss, to make each solution space of the multi-branch model orthogonal. We evaluate our approach on CIFAR-10, CIFAR-100 and SVHN against ℓ_∞ norm-bounded perturbations of size $\epsilon = 8/255$, respectively. Exhaustive experiments are conducted to show that our method goes beyond all state-of-the-art methods without any tricks. Compared to all methods that do not use additional data for training, our models achieve 67.3% and 41.5% robust accuracy on CIFAR-10 and CIFAR-100 (improving upon the state-of-the-art by +7.23% and +9.07%). We also outperform methods using a training set with a far larger scale than ours.

Index Terms—Deep neural network, Adversarial training, Model robustness, Multi-branch network.

I. INTRODUCTION

Deep Neural Networks (DNNs) have got great research attention and success in various application scenarios [1], [2]. However, it is found that DNNs are vulnerable to adversarial examples [3]. Leaving the model to produce the wrong output as the attacker wishes poses a significant security risk. Therefore, the vulnerability of DNNs to adversarial attacks is a critical topic in many areas with safety-vital nature, such as autonomous driving [4], and medical diagnostics [5].

Ever since Szegedy et al. [7] brought this problem to the forefront, many studies have sprung up on generating adversarial examples [8]–[10] and building defenses against

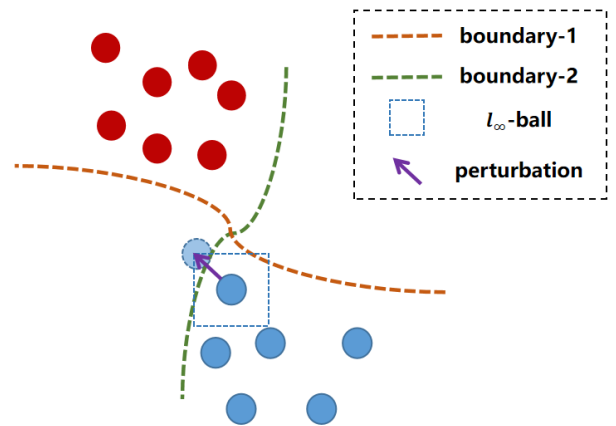


Fig. 1: The distribution of simplified binary classification models in the solution space, where blue and red dots represents two categories of data samples respectively.

these attacks [3], [11], [12]. Among many successful defenses, adversarial training became one of the mainstream and proved to be effective and practical. Madry et al. [11] first introduced projected gradient descent (PGD) to generate adversarial examples for training, achieving model robustness and invariance to input perturbations. As a standard for training adversarially robust models, a variety of modifications over their implementation have been proposed, such as Lipschitz regularization [13], [14], computationally efficient adversarial training [15], [16], utilization of unlabeled data [17], [18] and better trade-offs between robustness and natural generalization [19], [20].

However, in recent years, the improvement in robustness of the model seems to be less effective and practical. To further advance the model robustness, Carmon et al. [17], Najafi et al. [21] and Zhai et al. [22] pioneered the use of additional data (under both supervised and semi-supervised settings). Gowal et al. [23] trained a model on CIFAR-10 against ℓ_∞ perturbations of size $\epsilon = 8/255$ with a maximum enhancement of +8.73% robust accuracy when using additional unlabeled

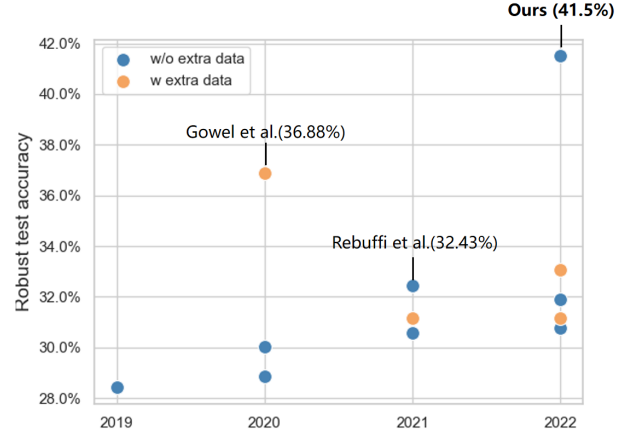
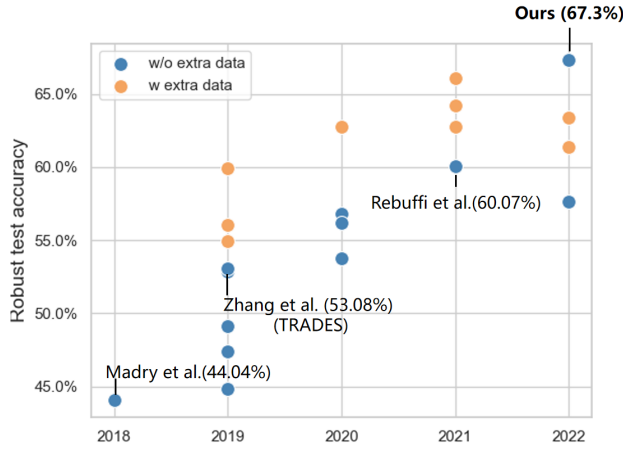


Fig. 2: Robust accuracy of models against AutoAttack [6] on CIFAR10 (left) and CIFAR100 (right) dataset.

data, uncovering the importance of additional datasets for robustness of the model obtained from adversarial training.

Although the methods listed above have proven to be very effective and continually advance the boundaries of model robustness, these works mainly focus on the data for adversarial training but the model. As we observed, they are all subject to the same restrictions, namely, that the solution space of the model leveraged is onefold and immutable, leaving it open to white-box attacks. It is also evident from the experimental observations that as one increases the scope limitation of perturbation, they produce a significant decrease in robust accuracy.

We revisit the issue of model robustness under attack from the perspective of solution (i.e. representation) space. As demonstrated in Fig. 1, a single classification boundary can be easily and successfully attacked by a perturbation bounded in the ℓ_∞ -ball, yet another classification boundary with a different distribution of solution space can still correctly classify the perturbed example. Therefore, we come up with the idea of using a neural network with several branches, each of which is a standard classification model (e.g. ResNet, WRN), to perform this integration of classification boundaries. To be more rigorous, the multi-branch structure we leverage is just a simple design paradigm for implementing multiple solution spaces, where the *branch* itself can stand for a broader concept, such as solution space. And it intuitively stands to reason that the more distinct these classification boundaries are, the more complementary they are, and the more effective they will work together as a whole. However, realizing solution space orthogonality is quite challenging because the process of optimizing network parameters using stochastic gradient descent is very uncertain. To make the different classification boundaries corresponding to the different branches vary widely, a novel loss function for supervision is necessary. Therefore, we propose branch-orthogonal loss, derived from the idea of using cosine similarity to evaluate the orthogonality between vectors, to let the newly trained branches optimize in a different direction from the already trained ones as much as

possible.

We propose *Branch Orthogonality adveRsarial Training* (BORT), a method to build robust models with the multi-branch structure that holds multiple orthogonal solution spaces, performing as a complementarity to the current prevailing data-centric design philosophy. Our method surpasses all current methods with or without additional datasets in robust accuracy, as listed in Fig. 2. To conclude, our contributions can be folded as follows:

- We revisit the model robustness from the perspective of solution space and reveal the effectiveness of multiple orthogonal solution spaces in improving the overall model robustness.
- A multi-branch model structure and its training pipeline are introduced to build models with state-of-the-art robustness against adversarial attacks. (Robust accuracies of 67.3% on CIFAR-10 and 41.5% on CIFAR-100 against ℓ_∞ perturbations AutoAttack, with an improvement of about +7.23% and +9.07% compared to the current state-of-the-art methods without extra training data.)
- We propose the branch-orthogonal loss, which provides supervision between multiple branches to achieve multi-branch solution space orthogonality.
- Despite of the need of training multiple branches, our method remains a moderate training cost since no extra data is used for better performance (i.e., neither generative models nor extra sizable datasets).

II. RELATED WORK

A. Adversarial Attacks

Research on the robustness of DNNs can be mainly divided into adversarial attacks and adversarial defenses, respectively. Adversarial attacks focus on crafting and adding imperceptible perturbation to the original input which makes the model misclassify. Adversarial attack techniques can be grouped into white-box and black-box methods. Under the setting of the white-box attack, the full architecture and parameters of DNNs are accessible while black-box attacks are not. Additionally,

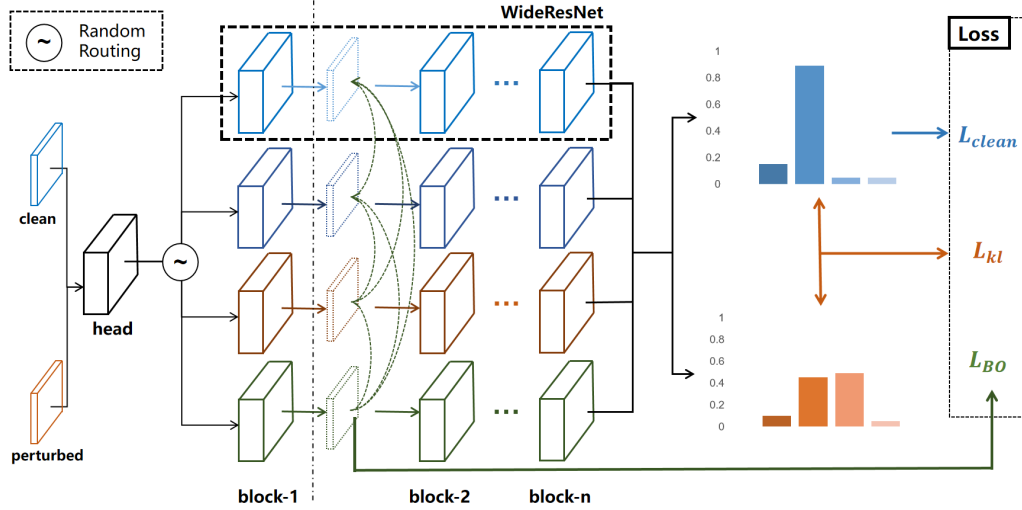


Fig. 3: Overview

attacks may be classified into two groups based on the sort of misclassification attackers aim to achieve: targeted attacks where the attacker aims to influence the misclassification label and untargeted attacks where misclassification occurs regardless of any particular label. Goodfellow, Shlens, and Szegedy [3] proposed the Fast Gradient Sign Method (FGSM) which generates adversarial examples. It was followed by R+FGSM [24], which adds a randomization step, and the Basic Iterative Method (BIM) [25], which takes multiple smaller gradient steps.

In this paper, we mainly focus on AutoAttack [6], which is widely recognized as a standard evaluation for model robustness combined with four complementary attacks, to test our method and make comparison with others.

B. Adversarial Training as a defense

On the opposite to the thriving development of adversarial attack methods, researchers also demonstrate great interest in creating various defense methods, which can be roughly divided into two different approaches, the certified defense, and the empirical defense. While the certified defense adopts optimization objective (loss functions or regularization terms) to provide some guarantee over robustness [26]–[28], the empirical defense utilizes adversarial samples during the training process.

Adversarial training is one of the notable defense methods as an empirical defense, which is also adopted in this paper to build robust models. And its basic idea can be expressed as a min-max optimization problem:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,y) \in X} \left[\max_{\delta \in [-\epsilon, \epsilon]} \ell(f_{\theta}(x + \delta), y) \right], \quad (1)$$

where the f_{θ} is a DNN parameterized by θ , X stands for the training dataset and ℓ represents the loss function. And perturbations δ are bounded into the ϵ -ball.

To solve the inner maximization problem, PGD [11] is a normally used and effective method to generate perturbations.

PGD updates perturbations on the clean data with a small step size for a regulated number of steps. The idea of performing attacks iteratively makes it a more powerful attack method over the Fast Gradient Sign Method (FGSM) [3] which only iterates once. Then, the generated adversarial data would be projected back onto a l_p -norm ball with radius ϵ , l_{∞} and l_2 in a normal case.

Besides from adversarial training, there are many promising non-certified defense methods that argue robustness, such as the defense distillation [29], the activation purging [30], the input transformation [31], and etc. However, many of these methods were then proved to essentially cause *obfuscated gradients* that give a false sense of security [32]. So that these defense methods can still be breached by reconstructing the gradient or by using stronger black-box attack methods. On the contrary, methods based on adversarial training such as PAT [11], TRADES [19] and MART [33] still yield good performance on many white-box and black-box attacks.

Recently, Gowal et al. [23] studied the effectiveness of multiple hyper-parameters and uncovered the sizable robust-accuracy gap between models trained solely on data from the original training set and those trained with additional data extracted from the “80 Million Tiny Images” dataset (80M-TI). Gowal et al. [34] followed this work by proposing that even the non-realistic data generated by generative models can also provide a considerable improvement in robustness. Rebuffi et al. [35] conducted extensive experiments and made a comparison between heuristics-driven augmentation techniques and generative models.

III. PROPOSED METHOD

We propose a novel framework, BORT, by introducing a multi-branch neural network and a new loss term for orthogonality learning. We first introduce the pipeline of our framework in Sec. III-A and then present our novel branch-orthogonality loss in Sec. III-B and our optimization formulation in Sec. III-C followed by the training pipeline in Sec. III-D.

Algorithm 1 Training Pipeline

Input: Model f , data x , labels y , num of branches K , number of epochs $epochs$, PGD algorithm $\text{PGD}()$

```
1: for  $k = 0$  to  $k = K$  do
2:    $\lambda_1, \lambda_2 \leftarrow 3$ 
3:   if  $k > 0$  then
4:      $f.\text{head.parameters}().\text{requires\_grad} \leftarrow \text{False}$ 
5:   end if
6:   for  $e = 0$  to  $e = epochs$  do
7:     if  $e > 40$  then
8:        $\lambda_1, \lambda_2 \leftarrow 1$ 
9:     end if
10:     $\hat{y} \leftarrow f^k(x)$ 
11:     $x' \leftarrow \text{PGD}(f^i, x, y)$ 
12:     $\hat{y}' \leftarrow f^k(x')$ 
13:    for  $j = 0$  to  $j = k - 1$  do
14:       $\mathcal{L}_{\text{BO}} += \frac{1}{k} |\cos\_sim(f_1^j(x'), f_1^k(x'))|$ 
15:    end for
16:     $\mathcal{L}_{\text{clean}} \leftarrow \text{CE}(\hat{y}, y)$ 
17:     $\mathcal{L}_{\text{KL}} \leftarrow \text{KL}(\hat{y}', \hat{y})$ 
18:     $loss \leftarrow \mathcal{L}_{\text{clean}} + \lambda_1 \mathcal{L}_{\text{KL}} + \lambda_2 \mathcal{L}_{\text{BO}}$ 
19:     $loss.\text{backward}()$ 
20:     $f^k.\text{update}()$ 
21:  end for
22: end for
```

a) Notation: $(x, y) \in \mathcal{X}$ is the natural dataset, and x' signifies perturbed examples. f^k denotes any of the K branches and f_i^k especially stands for the i -th ‘block’ (consistent to the definition in Zagoruyko and Komodakis [36]) of branch k . δ indicates perturbations bounded in the ϵ -ball, where ϵ represents the numerical limit of the perturbation.

A. Multi-branch Neural Network

The overview pipeline of our proposed method can be roughly concluded in Fig. 3. The clean and perturbed examples will first go through a shared head, consisting of a 2D convolutional layer. Then, the generated features will be randomly routed to one of four branches, each of which is an independent WideResNet model. We adopt the commonly used three settings with different model scales: WRN-28-10, WRN-34-10, and WRN-70-16, respectively. The possibility of routing to each branch is equal. Our multi-branch structure shares a greater model capacity and representation ability, while the inference time and computational cost stay consistent with an individual WRN model. The overall performance of our method compared to state-of-the-arts will then be exhaustively discussed in Sec. IV.

a) Randomness might be enough: Routing to one of K branches randomly is a thoroughly non-differentiable operation, so that white-box attacks that are based on optimization will be completely invalidated. To be more precise, every iteration for generating perturbations can only target a specific branch of the network, not the entire. Predictably, if we use some differentiable approach for branch selection, the multi-

branch network will still be very vulnerable, and the multi-branch design will then become redundancy.

B. Branch-orthogonal Loss

To guide the learning of multi-branch models, we heuristically propose an additional loss term to make the solution spaces of different branches orthogonal, which we believe will maximize the model robustness. When training the last branch (k -th in K branches) of our model, its mathematical form is as follows:

$$\begin{aligned} \mathcal{L}_{\text{BO}} &= \frac{1}{k-1} \sum_{j=1}^{k-1} |\cos_sim(f_1^j(x'), f_1^k(x'))| \\ &= \frac{1}{k-1} \sum_{j=1}^{k-1} \left| \frac{f_1^k(x') f_1^j(x')}{\max(\|f_1^k(x')\| \cdot \|f_1^j(x')\|, \epsilon)} \right|, \end{aligned} \quad (2)$$

where $\cos_sim$ denotes the cosine similarity, and ϵ is a sufficiently small and positive number.

C. Optimization Formulation

a) Loss for inner maximization: Retrospective of the mini-max optimization problem in adversarial training, the inner maximization is for generating adversarial examples x' from clean examples x . Here, we use the commonly used the cross-entropy loss to find adversarial examples as follows:

$$x' = x + \arg \max_{\delta \in [-\epsilon, \epsilon]} \text{CE}(f^k(x + \delta), y). \quad (3)$$

where CE denotes the cross-entropy loss.

b) Loss for outer minimization: The outer minimization problem is to allow the model to correctly classify adversarial examples. To obtain a better trade-off between the model robustness and its accuracy on clean data, we use the loss design proposed in TRADES [19]. In addition to the branch-orthogonal loss we introduced for orthogonality learning, the loss is composed of two parts: the first encourages the natural error (on clean data) to be minimized by applying the cross-entropy loss between the prediction of clean example $f^k(x)$ and labels y , and the second acts like a regularization term optimizing the difference between the prediction of clean example $f(x)$ and that of adversarial example $f^k(x')$. And there is a hyper-parameter balancing the importance of the two parts.

c) End-to-end optimization: The full version of loss that we use to optimize networks is presented here, combining all of the aforementioned loss terms. It consists of three distinct components, each of which serves a different purpose: the cross-entropy loss to fit our model to clean examples, the Kullback-Leibler loss to close the gap between prediction results for clean and perturbed examples, and our newly

proposed Branch-orthogonal loss to make the solution space of each branch orthogonal.

$$\mathcal{L}_{\text{clean}} = \text{CE}(f^k(x), y), \quad (4)$$

$$\mathcal{L}_{\text{KL}} = \text{KL}(f^k(x'), f^k(x)), \quad (5)$$

$$\mathcal{L}_{\text{BO}} = \frac{1}{k-1} \sum_{j=1}^{k-1} |\text{cos_sim}(f_1^j(x'), f_1^k(x'))|, \quad (6)$$

$$\mathcal{L}_{\text{BORT}} = \mathcal{L}_{\text{clean}} + \lambda_1 \mathcal{L}_{\text{KL}} + \lambda_2 \mathcal{L}_{\text{BO}}, \quad (7)$$

where KL denotes the Kullback-Leibler loss, λ_1 and λ_2 are tuning parameters balancing the importance among the three loss function.

D. Training Pipeline

The overall training process of our proposed method is illustrated in Algorithm 1, where the K branches of the network are trained separately in a serial fashion. We use the PGD algorithm to generate adversarial examples under the setting of $\epsilon = 8/255$, $\alpha = 0.01$ and steps = 10. The branch-orthogonal loss is calculated between the currently trained branch with all branches that have been trained previously then averaged with k . For example, when training branch No.2 (indexing from 0), branch-orthogonal loss will be calculated in branch No.0 with branch No.2, and No.1 with No.2.

IV. EXPERIMENT

A. Evaluation Setting

a) Datasets & Adversarial attacks: Three publicly available benchmark datasets are used. The CIFAR-10 dataset consists of 60,000 32x32 color images of 10 classes, with 6,000 images per class. There are 50,000 training images and 10,000 test images. The CIFAR-100 dataset is like CIFAR-10, except that it has 100 classes, each containing 600 images, of which 500 are training images and 100 are test images. The Street View House Number (SVHN) Dataset is derived from Google street view house numbers, each image contains a set of '0-9' Arabic numbers. The training set contains 73257 digits and the test set contains 26032 digits.

FGSM, PGD, and AutoAttack are utilized to evaluate our proposed method along with state-of-the-arts. Testing robustness against PGD is done in 40/100 iterations with the maximum $\epsilon = 8/255$ and step sizes of 0.01 for each iteration. FGSM attack also uses the same 8/255 bound for perturbation. We leverage the implementation of AutoAttack from torchattacks repository [42].

b) Model training details: We use WideResNets as our base models: WRN-28-10, WRN-34-10 and WRN-70-16 [36], respectively. Each branch is trained for 50 epochs using stochastic gradient descent (SGD) as the optimizer, with an initial learning rate of 0.1 that decays to 0.01 at the 40th epoch. The hyper-parameters are configured as $\lambda_1 = \lambda_2 = 3$ during the first 40 epochs, followed by $\lambda_1 = \lambda_2 = 1$ for the remaining 10 epochs.

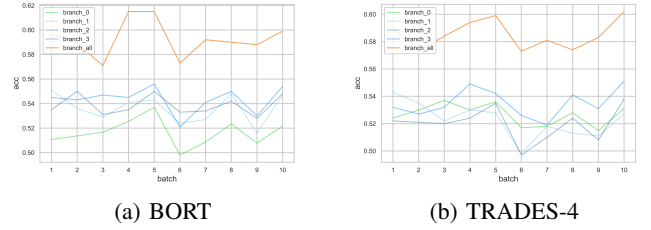


Fig. 4: Evaluation results for individual branches on evenly segmented test data batches of CIFAR-10. The model used here is WRN-28-10. Each individual branch is attacked with PGD25 and the complete network with PGD100.

B. Overall Performance

To illustrate the effectiveness of the proposed BORT, we train various models using this framework and evaluate their robustness against different adversarial attack algorithms. Furthermore, the proposed method is compared with different state-of-the-art approaches [23], [37]–[41]. The detailed experiment results are illustrated in Table I. For a fair comparison, only the results that do not use the additional dataset (80M-Ti for example) are presented in the table, although we still outperform the results using it. The 80M-Ti dataset has been taken down by the author team, given that it may contain offensive and biased content. Likewise, we do not try to train stronger models utilizing this dataset as additional training data.

Compared to all methods that do not use additional datasets, we obtain a maximum robustness improvement of about **+7.23%** in CIFAR-10, **+9.07%** in CIFAR-100 and **+1.00%** (WRN-28-10 solely) in SVHN over the highest results proposed. Our method also surpasses those trained with extra datasets: improvement of **+3.95%** and **+8.45%** in CIFAR-10 and CIFAR-100, respectively. In a longitudinal comparison of our method, it is consistent with the intuition that using a larger base model leads to an overall performance improvement. This improvement, however, is more minor than that provided by our proposed method.

With the same base model, we achieve the best results on robustness compared to all previous methods. It's worth mentioning that our method outperforms all current SOTA methods with base model WRN-70-16 by a significant gap of **+7.23%** (extra data SOTA **+3.95%**) in CIFAR-10 and **+9.07%** (extra data SOTA **+8.45%**) in CIFAR-100. Even with our lightest base model WRN-28-10, we still have a huge performance improvement of **+5.76%** (extra data SOTA **+4.76%**) in CIFAR-10 and **+8.4%** (extra data SOTA **+7.12%**) in CIFAR-100.

C. Ablation Studies

We study the effect of altering the number of branches on model overall robustness, results are listed in Table II. We evaluate the performance of single-branch, two-branch, and the complete four-branch model on the CIFAR-10 dataset facing adversarial attacks FGSM, PGD, and AutoAttack, respectively.

Dataset	Defense Method	Model	Clean	AutoAttack
CIFAR-10	Wu, Xia, and Wang [37]	WRN-28-10(RST-AWP)	88.3%	60.0%
	Gowal et al. [23]	WRN-70-16	85.3%	57.2%
	Rebuffi et al. [38]	WRN-28-10	86.1%	57.5%
	Rebuffi et al. [38]	WRN-34-10	86.2%	58.1%
	Rebuffi et al. [38]	WRN-70-16	87.3%	60.1%
	Schwag et al. [39] + 10M DDPM	WRN-34-10	86.7%	60.3%
	Pang et al. [40] + 1M DDPM	WRN-28-10	88.6%	61.0%
	Pang et al. [40] + 1M DDPM	WRN-70-16	89.0%	63.4%
	Jia et al. [41]	WRN-34-10(LAS-AWP)	87.7%	55.5%
	Jia et al. [41]	WRN-70-16	85.7%	57.6%
	BORT(Our)	WRN-28-10	87.5%	65.8%
	BORT(Our)	WRN-34-10	87.7%	66.3%
	BORT(Our)	WRN-70-16	88.3%	67.3%
CIFAR-100	Wu, Xia, and Wang [37]	WRN-34-10	60.4%	28.9%
	Gowal et al. [23]	WRN-70-16	60.9%	30.0%
	Rebuffi et al. [38]	WRN-28-10	63.0%	29.8%
	Rebuffi et al. [38]	WRN-70-16	65.8%	32.4%
	Schwag et al. [39] + 1M DDPM	WRN-34-10	65.9%	31.2%
	Pang et al. [40] + 1M DDPM	WRN-28-10	63.7%	31.1%
	Pang et al. [40] + 1M DDPM	WRN-70-16	65.6%	33.1%
	Jia et al. [41]	WRN-34-10	64.9%	30.8%
	BORT(Our)	WRN-28-10	63.7%	38.2%
	BORT(Our)	WRN-34-10	64.2%	39.0%
	BORT(Our)	WRN-70-16	63.7%	41.5%
SVHN	Wu, Xia, and Wang [37]	WRN-34-10	92.8%	58.7%
	Gowal et al. [23]	WRN-28-10	92.9%	57.3%
	Gowal et al. [23]	WRN-34-10	93.1%	58.0%
	Rebuffi et al. [38]	WRN-28-10	94.5%	57.3%
	BORT(Our)	WRN-28-10	93.1%	59.7%

TABLE I: Comparison of state-of-the-art adversarial defense methods with clean and perturbed-data accuracy on CIFAR-10, CIFAR-100, and SVHN dataset under AutoAttack with l_∞ perturbations of size 8/255.

Model	Clean	FGSM	PGD40	PGD100	AutoAttack
BORT-WRN-28-10-1	87.1%	52.0%	52.0%	51.9%	51.5%
BORT-WRN-28-10-2	86.1%	55.3%	56.7%	55.9%	59.5%
BORT-WRN-28-10-4	87.5%	64.7%	59.4%	59.3%	65.8%
BORT-WRN-70-16-1	87.3%	61.1%	55.8%	55.7%	54.1%
BORT-WRN-70-16-2	87.6%	66.2%	58.3%	57.9%	62.2%
BORT-WRN-70-16-4	88.3%	67.7%	60.5%	60.4%	67.3%

TABLE II: The performance of single-branch, two-branch and the complete four-branch model on the CIFAR10 dataset facing adversarial attacks FGSM, PGD and AutoAttack, respectively.

The robust accuracy improvement brought by adding more branches is obvious, which is independent of the base model used. It is intuitively consistent that we can achieve higher model robustness by combining multiple branches that hold orthogonally distributed solution spaces. This fact is also evident in the data listed in Fig. 4.

We are also inspired by the results listed in Fig. 4 to revisit BORT from a different perspective. Expectedly, PGD-100 will iterate 25 times on each branch separately. However, the robust accuracy, with PGD-100 to generate perturbations, obtained from evaluating the complete network is significantly higher than that obtained from evaluating individual branches with PGD-25. We assume that this is due to the random routing mechanism, where PGD does not iterate over a single

branch, but takes the results of iterations on other branches as the starting point for optimization, resulting in sub-optimal perturbations that are generated for performing an attack. The iterative optimization directions on different branches may also incur confrontations.

With our proposed branch-orthogonal loss, the complementarity of all branches is prone to be fully exploited by providing supervision to have the solution space of each branch orthogonal. Contrarily, in TRADES-4, the solution space distribution of each branch might be different, but is completely arbitrary and not guaranteed. By fixing the branches chosen at each inference, we evaluate the accuracy of each branch of the model trained by the BORT and TRADES, respectively. Fig. 4(b) shows that, if there is no Branch-orthogonal loss, each

Methods	Loss			Attacks				
	$\mathcal{L}_{\text{clean}}$	$\mathcal{L}_{\text{KL}}$	$\mathcal{L}_{\text{BO}}$	Clean	FGSM	PGD40	PGD100	AutoAttack
Vanilla	✓			93.8	8.2	0	0	0
PAT		✓(CE)		86.5	60.5	52.5	52.5	50.5
TRADES	✓	✓		86.6	58	51.1	51	49.8
TRADES-4	✓	✓		86.3	64.7	58.7	58.7	64.9
BORT(WRN-28-10)	✓	✓	✓	87.5	64.7	59.4	59.3	65.8
BORT(WRN-70-16)	✓	✓	✓	88.3	67.7	60.5	60.4	67.3

TABLE III: Ablation studies of loss terms. Vanilla: Models trained entirely with clean samples. TRADES-4: The same model structure as our method proposed, but the branches are trained according to TRADES [19](differ in loss function).

branch has nearly equivalent performance and appears to function similarly. In contrast, with Branch-orthogonal loss that plays a role of guidance, branch No.1/2/3 outperforms branch No.0 by about +2%, as illustrated in Fig. 4(a). Overall speaking, the robust accuracy of BORT relative to TRADES-4 is higher by about 0.9% under AutoAttack and 0.6% under PGD100 (Table III), with WRN-28-10 as the base model. For a once-iteration attack like FGSM, BO loss is of little help, resulting in consistent performance of TRADE-4 and BORT.

We also design an experiment to visualize the difference in the solution space of each branch of BORT and to show the effectiveness of this difference in confronting adversarial attacks. We perform transfer attacks in four branches, i.e., using perturbations generated by each branch to attack other branches. Visualized experiment results are shown in the form of heatmap as in Fig. 5. The lighter the color indicates that the attack is more likely to successfully interfere with the model classification results, and vice versa. A transfer attack yields a significantly lower attack success rate (i.e. higher robust accuracy), compared to each branch attacking itself. This is due to the orthogonality of the solution space of each branch, which prevents the perturbation from having an efficacious effect on the other branches, which is consistent with our motivation illustrated in Fig. 1. Another intriguing fact demonstrated in the results is that a less robust model(branch-1) may tend to generate weaker perturbations when performing transfer attacks.

To further validate effectiveness of proposed Branch-orthogonal loss, a loss term to encourage the solution space of each branch orthogonal, we retrain a model completely independent of the BORT model, through which we implement the transfer attacks on four branches separately. The experiment results along with more detailed numbers in Table IV. From the table, it is clear that perturbations from TRADES-0 have a comparable impact on branches 0 and 3, suggesting that the solution spaces of TRADES-0 and TRADES-3 without Branch-orthogonal loss for training are fairly similar. This similarity in the solution space will degrade the apparent boost brought by integrating multiple solution spaces. The finding further demonstrates that Branch-orthogonal loss can minimize transfer attacks across branches of BORT by making the solution space of each branch as orthogonal as feasible, whereas the solution space produced through TRADES training without

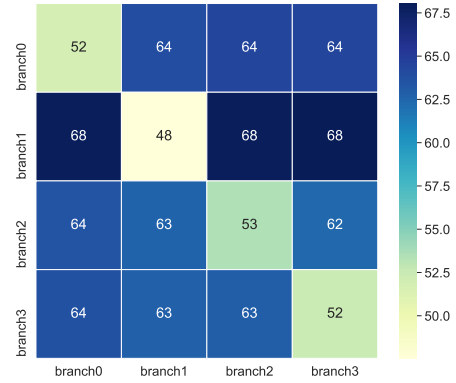


Fig. 5: Results of transfer attacks among the four branches. The numbers and their corresponding color representations robust accuracy against PGD-100 attack. The dataset and base model we use are CIFAR-10 and BORT-WRN-28-10 respectively.

	branch-0	branch-1	branch-2	branch-3
TRADES-0	51.95%	61.58%	62.73%	52.84%
BORT-0	51.81%	63.92%	64.39%	64.39%
BORT-1	67.85%	47.54%	67.86%	68.04%
BORT-2	63.50%	62.62%	53.46%	62.39%
BORT-3	63.66%	62.83%	62.91%	52.42%

TABLE IV: More detailed Transfer Attack results, where the results of the perturbations generated by a model on each branch are shown horizontally, and the results of the model under the perturbations generated by each model are shown vertically.

Branch-orthogonal loss is not at all guaranteed.

V. CONCLUSION

In this paper, we present BORT, the first adversarial training framework leveraging a multi-branch neural network, which improves network representation through multiple orthogonal branches while keeping inference time and computational cost from increasing, greatly improving model robustness against adversarial attacks. Through comprehensive experiments, we first illustrate that our proposed method goes well beyond the

current state-of-the-art by a noticeable gap without the need for additional training datasets. The efficiency of multi-branch structure and branch-orthogonal loss for enhancing overall robustness was then discussed. Our method achieve top-rank performance on RobustBench under AutoAttack.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Adv. Neural Inf. Process. Syst.*, vol. 25, 2012.
- [2] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath *et al.*, “Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups,” *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 82–97, 2012.
- [3] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” *arXiv preprint arXiv:1412.6572*, 2014.
- [4] K. Wang, F. Li, C.-M. Chen, M. M. Hassan, J. Long, and N. Kumar, “Interpreting adversarial examples and robustness for deep learning-based auto-driving systems,” *IEEE Trans. Intell. Transp. Syst.*, 2021.
- [5] J. De Fauw, J. R. Ledsam, B. Romera-Paredes, S. Nikolov, N. Tomasev, S. Blackwell, H. Askham, X. Glorot, B. O’Donoghue, D. Visentin *et al.*, “Clinically applicable deep learning for diagnosis and referral in retinal disease,” *Nat. Med.*, vol. 24, no. 9, pp. 1342–1350, 2018.
- [6] F. Croce and M. Hein, “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks,” in *Int. Conf. Mach. Learn.* PMLR, 2020, pp. 2206–2216.
- [7] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” *arXiv preprint arXiv:1312.6199*, 2013.
- [8] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, “Deepfool: a simple and accurate method to fool deep neural networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2574–2582.
- [9] N. Carlini and D. Wagner, “Towards evaluating the robustness of neural networks,” in *IEEE Symp. Secur. Privacy (SP)*, 2017. Ieee, 2017, pp. 39–57.
- [10] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, “Synthesizing robust adversarial examples,” in *Int. Conf. Mach. Learn.* PMLR, 2018, pp. 284–293.
- [11] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” *arXiv preprint arXiv:1706.06083*, 2017.
- [12] A. Jaddi, M. J. Shafiee, M. Karg, C. Scharfenberger, and A. Wong, “Learn2perturb: an end-to-end feature perturbation learning to improve adversarial robustness,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1241–1250.
- [13] M. Cisse, P. Bojanowski, E. Grave, Y. Dauphin, and N. Usunier, “Parseval networks: Improving robustness to adversarial examples,” in *Int. Conf. Mach. Learn.* PMLR, 2017, pp. 854–863.
- [14] Z. Yan, Y. Guo, and C. Zhang, “Deep defense: Training dnns with improved adversarial robustness,” *Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [15] A. Shafahi, M. Najibi, M. A. Ghiasi, Z. Xu, J. Dickerson, C. Studer, L. S. Davis, G. Taylor, and T. Goldstein, “Adversarial training for free!” *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [16] E. Wong, L. Rice, and J. Z. Kolter, “Fast is better than free: Revisiting adversarial training,” *arXiv preprint arXiv:2001.03994*, 2020.
- [17] Y. Carmon, A. Raghunathan, L. Schmidt, J. C. Duchi, and P. S. Liang, “Unlabeled data improves adversarial robustness,” *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [18] J.-B. Alayrac, J. Uesato, P.-S. Huang, A. Fawzi, R. Stanforth, and P. Kohli, “Are labels required for improving adversarial robustness?” *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [19] H. Zhang, Y. Yu, J. Jiao, E. Xing, L. El Ghaoui, and M. Jordan, “Theoretically principled trade-off between robustness and accuracy,” in *Int. Conf. Mach. Learn.* PMLR, 2019, pp. 7472–7482.
- [20] J. Zhang, X. Xu, B. Han, G. Niu, L. Cui, M. Sugiyama, and M. Kankanhalli, “Attacks which do not kill training make adversarial learning stronger,” in *Int. Conf. Mach. Learn.* PMLR, 2020, pp. 11 278–11 287.
- [21] A. Najafi, S.-i. Maeda, M. Koyama, and T. Miyato, “Robustness to adversarial perturbations in learning from incomplete data,” *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [22] R. Zhai, T. Cai, D. He, C. Dan, K. He, J. Hopcroft, and L. Wang, “Adversarially robust generalization just requires more unlabeled data,” *arXiv preprint arXiv:1906.00555*, 2019.
- [23] S. Goyal, C. Qin, J. Uesato, T. Mann, and P. Kohli, “Uncovering the limits of adversarial training against norm-bounded adversarial examples,” *arXiv preprint arXiv:2010.03593*, 2020.
- [24] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, “Ensemble adversarial training: Attacks and defenses,” *arXiv preprint arXiv:1705.07204*, 2017.
- [25] A. Kurakin, I. J. Goodfellow, and S. Bengio, “Adversarial examples in the physical world,” in *Artif. Intell. Saf. Secur.* Chapman and Hall/CRC, 2018, pp. 99–112.
- [26] M. Hein and M. Andriushchenko, “Formal guarantees on the robustness of a classifier against adversarial manipulation,” *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [27] A. Raghunathan, J. Steinhardt, and P. Liang, “Certified defenses against adversarial examples,” *arXiv preprint arXiv:1801.09344*, 2018.
- [28] A. Sinha, H. Namkoong, and J. C. Duchi, “Certifiable distributional robustness with principled adversarial training. corr. abs/1710.10571,” *arXiv preprint arXiv:1710.10571*, 2017.
- [29] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, “Distillation as a defense to adversarial perturbations against deep neural networks,” in *IEEE Symp. Secur. Privacy (SP)*, 2016. IEEE, 2016, pp. 582–597.
- [30] G. S. Dhillon, K. Azizzadenesheli, Z. C. Lipton, J. Bernstein, J. Kossaifi, A. Khanna, and A. Anandkumar, “Stochastic activation pruning for robust adversarial defense,” *arXiv preprint arXiv:1803.01442*, 2018.
- [31] C. Guo, M. Rana, M. Cisse, and L. Van Der Maaten, “Countering adversarial images using input transformations,” *arXiv preprint arXiv:1711.00117*, 2017.
- [32] A. Athalye, N. Carlini, and D. Wagner, “Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples,” in *Int. Conf. Mach. Learn.* PMLR, 2018, pp. 274–283.
- [33] Y. Wang, D. Zou, J. Yi, J. Bailey, X. Ma, and Q. Gu, “Improving adversarial robustness requires revisiting misclassified examples,” in *Int. Conf. Learn. Represent.*, 2019.
- [34] S. Goyal, S.-A. Rebuffi, O. Wiles, F. Stimberg, D. A. Calian, and T. A. Mann, “Improving robustness using generated data,” *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 4218–4233, 2021.
- [35] S.-A. Rebuffi, S. Goyal, D. A. Calian, F. Stimberg, O. Wiles, and T. Mann, “Fixing data augmentation to improve adversarial robustness,” *arXiv preprint arXiv:2103.01946*, 2021.
- [36] S. Zagoruyko and N. Komodakis, “Wide residual networks,” *arXiv preprint arXiv:1605.07146*, 2016.
- [37] D. Wu, S.-T. Xia, and Y. Wang, “Adversarial weight perturbation helps robust generalization,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 2958–2969, 2020.
- [38] S.-A. Rebuffi, S. Goyal, D. A. Calian, F. Stimberg, O. Wiles, and T. A. Mann, “Data augmentation can improve robustness,” *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 29 935–29 948, 2021.
- [39] V. Sehwag, S. Mahlouiifar, T. Handina, S. Dai, C. Xiang, M. Chiang, and P. Mittal, “Robust learning meets generative models: Can proxy distributions improve adversarial robustness?” *arXiv preprint arXiv:2104.09425*, 2021.
- [40] T. Pang, M. Lin, X. Yang, J. Zhu, and S. Yan, “Robustness and accuracy could be reconcilable by (proper) definition,” *arXiv preprint arXiv:2202.10103*, 2022.
- [41] X. Jia, Y. Zhang, B. Wu, K. Ma, J. Wang, and X. Cao, “Las-at: Adversarial training with learnable attack strategy,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 13 398–13 408.
- [42] H. Kim, “Torchattacks: A pytorch repository for adversarial attacks,” *arXiv preprint arXiv:2010.01950*, 2020.